SafeStreet

AI-Powered Community Incident Reporting Platform

Deliverable 5 — Written Reflection

Three Architectural Decisions, Tradeoffs, and What I Would Change

Introduction

Over the course of building SafeStreet, an AI-powered community incident reporting platform, our team made dozens of small implementation choices and three that stand out as genuinely architectural: which large language model provider to integrate behind our AI proxy, how to structure global state on the frontend, and how to configure our Docker-based development environment. Each of these decisions looked straightforward when we made it, and each one taught us something we did not fully appreciate until we were living with the consequences, often while debugging at unexpected hours. This reflection walks through those three decisions, the tradeoffs we did and did not see coming, and what we would genuinely do differently if we were starting the project again today.

1. AI Provider Selection: Google Gemini vs. the Specified Claude/GPT-4o

The project brief specified OpenAI's GPT-4o or Anthropic's Claude as the required LLM providers for the AI proxy module. We built the entire ai-proxy.service.ts layer around Google Gemini instead, using the gemini-flash-latest model through the @google/generative-ai SDK. This was not an oversight so much as a pragmatic choice made under time pressure: Gemini's free-tier quota was generous enough to let us iterate quickly through many rounds of prompt design without worrying about burning through paid API credits, its responseSchema feature gave us clean, JSON-schema-constrained output with very little extra plumbing, and getting a working key took minutes rather than requiring a billing setup. For a project where we needed to prototype four distinct AI features — classification and routing, duplicate detection, response drafting, and hotspot prediction — being able to move fast mattered more, in the moment, than following the letter of the spec.

Looking back, we think this was the right call for the development phase but not a decision we would leave unexamined for a final submission. The tradeoff is really about audience and accountability rather than raw technical capability: Gemini, Claude, and GPT-4o are all more than capable of the structured classification and text-generation tasks we needed, so the technical risk of using Gemini was low. The real cost is that the brief explicitly named two other providers as a "Mandatory Constraint," and quietly substituting a third without flagging it is the kind of small deviation that can look like carelessness even

when the underlying engineering is sound. If we were doing this again, we would handle it one of two ways: either swap the SDK call in AiProxyService to Anthropic's or OpenAI's client early, before the rest of the codebase built up dependencies on Gemini-specific behaviour like its particular responseSchema format, or — if a provider swap were not practical under the deadline — raise the substitution with the instructor explicitly and in writing before submission, rather than only documenting it honestly in the reports after the fact. The lesson I am taking from this is less about Gemini specifically and more about how easy it is to let a reasonable development-time shortcut quietly become the final architecture unless someone deliberately revisits it before submission.

2. A Single Global Zustand Store for All Frontend State

On the frontend, we centralised essentially all shared state — the authenticated user, the full incidents collection, real-time WebSocket listeners, and SLA alerts — into one Zustand store, useIncidentStore. Every page component that needed any of this data imported the same hook. At the time this felt like an obviously good decision: it meant we never had to prop-drill authentication state down through nested components, every dashboard could subscribe to the same live incidents array without duplicating fetch logic, and wiring up the WebSocket listeners once in the store meant CitizenDash, WorkerDash, SupervisorDash, and AdminDash all got real-time updates for free just by calling connectRealtime() rather than each page managing its own socket connection.

The tradeoff became more visible as the store grew. By the time we had auth actions, incident-fetching actions, and real-time event handlers all living in the same file, it was no longer obvious at a glance which piece of state a given component actually depended on, and the store itself became a file we had to hold a lot of context in our heads to safely edit. There is also a subtler performance tradeoff: because everything lives in one store, a component that only cares about the current user re-renders on updates to the incidents array too, unless you are careful with selectors — something we did not rigorously apply throughout the app. For a project of this size, that cost never became a real problem we could measure, but it is the kind of thing that would start to matter if SafeStreet grew past four dashboards and a couple dozen components.

If we rebuilt this today, we would keep Zustand as the state management library — it genuinely was simpler than introducing Redux or leaning entirely on React Context, and the brief specifically noted Redux was not required — but we would split the single store into feature-specific slices: one for authentication, one for the incidents collection and its real-time listeners, and possibly a separate one for UI-only state like modal visibility. Zustand supports this pattern natively without much extra ceremony, and it would have made it much clearer, especially for a teammate joining the codebase later, which actions and state belong together. This is a case where the initial simplicity of "just one store" was the right call to get moving quickly, but not the shape we would want to defend as the final architecture of a larger application.

3. Docker Development Environment: Bind Mounts and the Compiled-Output Conflict

The most viscerally frustrating architectural lesson of this project came from our Docker Compose setup, and specifically from a conflict between two goals that seemed compatible on paper: wanting the backend container to run a production-style compiled build, and wanting to bind-mount our local source code into the container so we would not need to rebuild the image every time we changed a line of backend code. Our first docker-compose.yml built the NestJS app with npm run build inside the image and ran the compiled dist/main.js, while also bind-mounting ./backend:/app for convenience. What we did not realise until the container actually failed to start was that the bind mount replaces the entire /app directory at container start, including the dist/ folder that had just been compiled during the image build — so the container booted with no compiled output to run at all, and NestJS immediately crashed with Cannot find module '/app/dist/main'.

Our honest first reaction was confusion rather than immediate understanding — the build logs clearly showed a successful compilation step, so it was not obvious at first that a completely separate part of the same compose file was silently undoing that work moments later. It took walking through exactly what a bind mount does, layer by layer, to realise the two configuration choices were fundamentally incompatible as written: you cannot both compile something into an image and then mount over the exact directory that output lives in, and expect the output to survive. Once we understood the actual mechanism, the fix was conceptually simple — switch the backend to run in NestJS's watch mode (nest start --watch) instead of a compiled build, so there is no dist/ folder for the bind mount to clobber in the first place, and live edits are picked up by the watcher instead of requiring a rebuild.

If we were setting up the development environment again from the start, we would think about "does this service run from a compiled build or from live-reloaded source" as an explicit, upfront design decision for every containerised service, rather than something that gets bolted on after the fact once a convenience feature is added. I would also be more deliberate about only bind-mounting the specific subdirectories a service actually needs to watch, and documenting clearly, in the compose file itself, which services are running in which mode and why — since this exact class of bug (a convenience mount silently breaking a separate build step) is easy to reintroduce accidentally later even after it has been fixed once. More broadly, this was a useful reminder that Docker's layering and mounting behaviour is not always intuitive from reading a compose file top to bottom, and that verifying assumptions about what actually ends up inside a running container is worth doing early, before chasing what looks like an application-level bug.

Conclusion

Across all three of these decisions, the common thread is that the choice which felt fastest or simplest in the moment — using whichever AI provider was easiest to set up, centralising all frontend state in one

place, and bind-mounting source code for convenience — was rarely the wrong instinct to have early in development, but each one needed a second, more deliberate pass before I would call it a finished architecture. Given more time, I would revisit the AI provider decision explicitly rather than letting it stand by default, split the frontend store along feature boundaries before the codebase grew any further, and treat the "compiled build vs. live-reload" question as a decision to make consciously for every service rather than something to discover through a failing container. None of these were mistakes in the sense of being obviously wrong choices at the time; they were reasonable tradeoffs that simply needed to be revisited as the project matured, which is itself the main lesson I am taking away from building SafeStreet.

End of Deliverable 5 — Written Reflection.